Git | Github

Git is the tool you use, and GitHub is the place where you store the work you did with that tool.

Git is software that runs locally on your computer. It doesn't need the internet to work.
It tracks every single change you make to your files.
If you make a mistake and your code breaks, you can "roll back" to a previous version that actually worked.

GitHub is an online platform (a website) that hosts projects managed by Git.
it acts as a central location in the cloud where you "push" your local Git work.
It allows you to backup your code, share it with others, and collaborate on the same project with developers across the world.

Git Commands

Setup & Starting

These are used when you start a new project or join an existing one.

git init: Initializes a brand new local repository in your current folder.
git clone <url>: Downloads an existing project from GitHub to your computer.
git config --global user.name "Your Name": Sets your name for commits.
git config --global user.email "email@example.com": Sets your email.

The Daily Workflow

You will use these commands 90% of the time while coding.

git add <file> or git add
Moves changes to the Staging Area. Think of it as "picking" the files you want to save.
git commit -m "Your message"
Saves the staged changes into the Local Repository (your history).
git push origin <branch-name>
Sends your local commits to GitHub.

Checking Your Work

To see what is happening in your repo.

git status: Shows which files are modified, staged, or untracked.
git log: Displays the history of all commits (who did what and when).
git diff: Shows the exact lines of code you changed compared to the last save.

Branching (Feature Management)

Use this so you don't break the main code while experimenting.

git branch: Lists all local branches.
git checkout -b <branch-name>: Creates a new branch and switches to it.
git switch <branch-name>: Moves back to an existing branch (like main).
git merge <branch-name>: Combines changes from another branch into your current one.

Syncing with GitHub

If you are working on multiple computers or with a team.

git remote add origin <url>: Connects your local repo to
git pull origin <branch-name>: Fetches the latest code from GitHub and merges it into your local files.
git fetch: Downloads history from GitHub without changing your local files yet.

Tip

Always run git status before you commit anything.
It prevents you from accidentally saving files you didn't mean to include
(like temporary test files or .env secrets).

.gitignore

A text file that tells Git which files or folders should not be tracked or uploaded to GitHub.
It keeps your repository clean and secure.
You should always ignore heavy folders like node_modules and sensitive files like .env (which contains your passwords/secrets).

git stash

Think of this as a "Pause" button for your code.
It temporarily hides your uncommitted changes so you can work on something else.
If you are halfway through a feature but need to fix an urgent bug on another branch, you stash your work, fix the bug, and then pop your work back later.
git stash (to hide) and git stash pop (to bring back).

Fixing Mistakes

Commands used to undo actions without losing your progress.

git reset <file>: This "unstages" a file. If you accidentally ran git add on a file you didn't want to save, this command reverses it.
git commit --amend: This lets you change your last commit. Use it if you made a typo in your commit message or forgot to add a small change.

README.md

The "Front Page" of your project on GitHub.
A Markdown file that explains what your project does.
Project description, installation steps, and the tech stack (like Node.js, Prisma, etc.).
It is the first thing employers or other developers see when they visit your GitHub profile.

Pull Requests (PRs)

The standard way to collaborate and merge code on GitHub.
Instead of pushing code directly to the main branch, you tell the team: "I have finished this feature, please review it and merge it".
It allows for "Code Reviews" where others can check for errors before the code becomes part of the final product.

Working Directory

The local folder where you are currently editing your files.

Staging Area (Index)

A "waiting room" or temporary area where you prepare changes before they are officially saved.

Commit as a "Snapshot"

Think of a commit not just as a "save," but as a snapshot of the entire project at a specific point in time.

Repository (Repo)

The "history book" for your project containing all files and the record of changes .

Why use branches?

To allow for parallel development and feature isolation so you don't break the main codebase while experimenting.

What is a Merge Conflict?

This happens when two people change the same line of code; you must resolve it manually before finishing the merge.